Manabot: Self-Play Reinforcement Learning for *Magic: The Gathering* — A Working Report

Jack Heart
Loopflow Studio

October 21, 2026

Written by AI. Not yet intended for distribution.

Abstract

We are training agents to play *Magic: The Gathering* by self-play. Part I says what we have demonstrated so far and what our failures taught us. Part II is a recipe: each component of the system, what it does, the work it draws on, and where we depart from it. Part II assumes the reader already knows why reinforcement learning for games is hard; Appendix A maps those difficulties to the literature for anyone who wants that framing. At small scale, on one fixed matchup, self-play now learns, and the rules engine is no longer a bottleneck. Search-as-teacher did not compound, and no single piece of our adapted Ataraxos recipe has yet shown a benefit on its own. The agents we have played still feel like beginners.

Part I
What we have demonstrated

1 Status

Manabot trains one network to play both sides of a single matchup, UR Lessons versus GW Allies from the *Avatar: The Last Airbender* set. Training uses self-play from terminal reward with an adaptation of the Ataraxos update [Sokota et al., 2026]. The rules engine, managym, is a deterministic, forkable Rust implementation of a two-player rules subset. The largest run used about 51M learner decisions on one L4 GPU. The agents beat scripted bots that avoid the most obvious blunders. When we have played them ourselves, they feel like beginners. We do not yet have a rating system that compares against human play.

2 What works

Finding	Evidence
A 10,000-update self-play cohort improves, mostly by learning to pilot Lessons.	Mean win rate against scripted greedy on monitoring deals rose from 30.3% to 54.3%. Allies saturated near 87–88% by update 1,240; Lessons rose from 5/150 to 32/150. The held-out comparison is pending.
Batching many games per update is a large, reliable speedup.	At equal experience, the 64×2 model trained in 109–111 minutes with 4 streams and 4.8–5.2 with 256. The smallest model learned as well as larger ones at this horizon; two seeds cannot rank sizes.
Collection dominates training time.	Collection was 82.5% of active time in one profiled run and 62% in another; policy calls were ≈87% of collection.
Opponent choice drives the measured gap between decks.	Switching the scripted opponent's deck moved Lessons results by 40–60 points on five deals.
Ataraxos pieces have not yet shown a benefit individually.	Actor-only advantage filtering: +3.33 points, 95% interval $[-4.33, +11.33]$, at 1.45× wall time. Value-token and pooling variants unresolved.
The encoding no longer hides decision-relevant differences.	The learnability search found 0 clean input collisions.
Largest run.	64×2 model, 64 streams, one L4: ≈51M learner decisions, 100,000 iterations (≈800k Adam steps).

Two of these are demonstrations rather than hopes. First, self-play from win/loss alone produces steady improvement on a hidden-information game with variable, object-referential actions. Second, the engine is fast enough that the rules account for about 4.5% of collection time, so further speed comes from inference batching, not from the simulator.

3 What failed, and what it taught us

What we tried or believed	What happened	What we took from it
Shaped rewards to help credit assignment	Terminal-only reward beat them by about 15 points	Use terminal reward; shaping installs a prior the deck may not want
Learning from search, KataGo-style: distill search, then iterate	Distillation beat a weak PPO baseline, but round 1 of expert iteration lost 74–26 to round 0	The loop compounds only if search clearly beats its own rollout policy. Price the label source, not the algorithm. Hard targets also cannot represent mixed strategies
Search win rate as evidence of good play	Search beat random 99% of the time, yet failed every control scenario; random beat it on 3 of 5	Win rate hides incompetence. Score decisions with known-correct scenarios
"Known decklists make determinization valid"	Wrong in principle: the opponent's hand is still a private choice	Hidden information changes the basic approach; Stratego, not Go, is the closer precedent
Calling the control failures strategy fusion	A review judged it premature; random rollouts rarely execute "wait, then act" even face-up. The isolating test was designed but never run	Name a mechanism only when an experiment isolates it
Matchup tables as deck measurements	They measured the pilot and card quality: four *Ancestral Recall* moved a result from 23.7% to 78.25%	A matchup number is joint in decks, pilot and a few strong cards
Unbalanced evaluation	Going first wins 94% between random players on a creature deck	Balance seats and decks in every evaluation
Hand-transcribed cards	20 of 55 were wrong	Card data needs CI against its source of truth
"The engine is the bottleneck"	Inference took 97% of step time once the engine was in Rust	Profile before optimizing; batch inference
Ataraxos pieces one at a time	Actor-only filtering: +3.33 points, interval $[-4.33, +11.33]$, at 1.45× wall time	Pieces may only matter together or at scale; that is now the main methodological question

4 Not yet demonstrated

- Strength against humans, or a rating that can be compared with human play.
- Play beyond one matchup. A ten-deck world is being built.
- A strength gain from inference-time search or learned beliefs over the policy alone.
- An effect of curriculum (mirrors versus cross-deck play, opponent mix). The held-out comparison for the main self-play cohort is also pending.
- Durable progress against past checkpoints and exploiters, beyond scripted anchors that are now saturating.

How we got here. We moved from KataGo-style learning from search to Ataraxos-style self-play, as summarized below.

When	Approach	What happened
2024 – mid 2026	PPO with shaped rewards on toy decks	The decks had no instants, removal or counterspells, so there was little strategy to learn. Shaping lost to terminal reward.
July 2026	KataGo-shaped: determinized Monte Carlo search, distillation, expert iteration	Search was strong and cheap; expert iteration regressed; the pilot failed every control scenario.
July 2026	Public-belief design, belief sampler, determinized PUCT	Strategy fusion suspected, never isolated; rollout pathology more likely.
October 2026	Ataraxos-shaped self-play; beliefs and search at inference	Self-play learns at small scale. The open questions moved to curriculum and evaluation.

Part II
Methods: the recipe and its influences

5 Overview

This part assumes the reader knows why game RL is hard. It describes what each component does, credits the work it draws on, and says where we depart from that work. Table 1 summarizes.

Table 1: The recipe at a glance.

Component	What we do	Influences	Code
Rules engine	Deterministic Rust engine; exact clone/fork; per-player observations; cards as typed programs run by a generic interpreter	Fast forkable simulators for search and self-play [Silver et al., 2018, Wu, 2019]; determinization for *Magic* [Cowling et al., 2012b]	`managym/src`
Collection	Many games in lockstep, batched inference, both decks and seats, current-self opponent, weights frozen per batch	Synchronous batched actors [Mnih et al., 2016, Espeholt et al., 2018]; high-throughput self-play [Sokota et al., 2026]	`managym/src/ python`, `manabot/sim/ rollout.py`
Network	Object-centric encoder with attention; card programs through a GRU; each legal action scored with its focus objects; win/draw/loss head	Transformers [Vaswani et al., 2017]; entity encoders and per-object action selection [Vinyals et al., 2019, 2015]	`manabot/model`
Update	Clipped surrogate, KL to collecting policy, annealed magnet KL, coordinated schedules, advantage filtering, EMA	PPO [Schulman et al., 2017]; TRPO [Schulman et al., 2015]; R-NaD [Perolat et al., 2021, 2022]; MMD [Sokota et al., 2023]; Ataraxos [Sokota et al., 2026]	`manabot/training/ ataraxos.py`, `references.py`
Credit assignment	Terminal reward; GAE with separate policy and value λ	TD(λ) [Sutton, 1988]; GAE [Schulman et al., 2016]; shaping theory [Ng et al., 1999]	`manabot/training/ ataraxos.py`
Beliefs	Hand-count model trained by supervised learning on true hidden state from self-play	Ataraxos belief network; public belief states as theory [Brown et al., 2020]	`manabot/belief`

Component	What we do	Influences	Code
Local search	Belief-sampled worlds, frozen-policy continuations, one MMD step	MMD [Sokota et al., 2023]; Ataraxos search; PIMC critique [Frank and Basin, 1998, Long et al., 2010]	`manabot/sim/local_update.py`, `local_sampling.py`
Evaluation	Seat- and deck-balanced head-to-head Tournaments, population ratings, competency scenarios, learnability checks, preregistration	Re-evaluating evaluation [Balduzzi et al., 2018]; exploiters [Vinyals et al., 2019]; seed variance [Henderson et al., 2018]	`manabot/arena`, `manabot/verify`

6 The rules engine

Rules. managym implements a two-player subset of the *Comprehensive Rules*: priority, the stack, combat, triggered and activated abilities, a simplified layer system for continuous effects, mid-resolution choices, and state-based actions. Replacement and prevention effects, full layer dependencies and mulligans are absent. Cards are authored as typed programs rather than parsed from Oracle text or hand-coded per card name. A generic interpreter executes them and refuses any program it cannot execute. Rules behavior is pinned by conformance fixtures, and card data is checked against a committed Scryfall snapshot. That check exists because a 2026 audit found 20 of 55 hand-transcribed cards wrong.

Information boundaries. At each decision the engine gives each player an observation of only what that player can see, plus the legal actions and the objects each one refers to. Viewer-equivalence, replay and leakage checks are hard gates. An early bug let the ordering of a hidden card pool change search results between states that should have looked identical to the player. The observation and action encoding is versioned as a *world*. Checkpoints load only in the world they were trained in, and a nightly learnability check looks for encodings that make two different decisions indistinguishable. The most recent broad search found none.

How the engine got fast, and why it is not on a GPU. The engine began as a Python prototype. Coding agents translated it into C++, and we then rewrote that C++ by hand. A profiling pass in January 2026 raised C++ throughput from 16k to 71k steps/s. The C++ design used pointer-heavy objects with tangled ownership. In March 2026 we ported to Rust with a data-oriented design: arenas with typed indices, enums with exhaustive matching, and one flat game state, which also makes exact cloning cheap. Vectorization and observation encoding moved into Rust with zero-copy buffers, taking environment-only throughput from 24k to 183k steps/s on an Apple M4 Max. With inference in the loop, throughput fell to 2.0k steps/s, and inference took 97% of step time. Later batching work raised in-loop inference from 2k to 24.5k observations/s.

In the current profile, policy calls are about 87% of collection time: 51% for the opponent and 36% for the learner. Engine stepping is about 4.5%. By Amdahl's law, even an infinitely fast engine would save at most about 4.5% of collection time. *Magic*'s rules are also branchy — the stack, triggers, variable action lists — which suits GPUs poorly, and a GPU port would have to re-implement every card. Stratego's fixed board and simple rules made a CUDA simulator a cheap choice for Ataraxos; they do not here. A GPU simulator's main advantage is very large co-located batches. Our calls average only about 11 positions across 64 games, so we pursue the same gain on CPU first: more concurrent games, double-buffering, and merging the learner's and opponent's

inference calls. Search changes this calculation. If search returns to training, forking becomes engine work again, and we would re-profile.

7 Network

A single network encodes every visible object — players, cards, permanents — and mixes them with attention [Vaswani et al., 2017]. Cards enter as tokens of their typed programs, encoded by a GRU, alongside explicit features; the intent is to embed what a card does, not just its name [Bertram et al., 2024]. Each legal action is scored together with the objects it refers to, so casting, targeting, attacking, blocking and paying costs all go through one action head. Unlike a pointer network [Vinyals et al., 2015], scoring is per legal action rather than per input position. A value head predicts win, draw or loss. The baseline model is 64 wide with 2 attention layers; at our current horizon the smallest model learned as well as larger ones.

8 Self-play collection

Many games step in lockstep, so one inference call serves many positions; the Rust vector environment writes observations into shared buffers without copying. The learner plays both decks and both seats against a current copy of itself, using a cross-balanced matchup schedule. Weights are frozen within each collection batch, but a game may span updates. For every row the collector stores the acting player's observation, the chosen action, and the full collection distribution over legal actions, which the update's KL terms need. Terminal outcomes supply labels, never policy inputs. Opponent actions are stepped between learner decisions and receive no gradient.

Two lessons shaped this. Batching was the largest single speedup: at equal experience, 256 streams trained the baseline model about 20 times faster than 4 streams. And in-loop inference, not the engine, sets the pace; calls still average only about 11 positions across 64 games, because the learner and opponent are separate calls and games wait on one another.

9 The update

We adapted the move-network update of Ataraxos [Sokota et al., 2026], which follows DeepNash [Perolat et al., 2022] in treating stability as the central problem. Stratego's separate setup network has no counterpart in *Magic*. The update is PPO-family. Table 2 compares it with our earlier PPO recipe, which remains as a control. Each addition is a separate switch so it can be tested on *Magic* rather than adopted on the paper's authority.

	Earlier PPO recipe	Ataraxos move recipe (baseline)
Clipped ratio surrogate	yes ($\epsilon = 0.1$)	yes ($\epsilon = 0.2$)
KL to collecting policy	no	yes (0.1)
KL to fixed magnet ρ	no	yes, $\tau = 0.05/t^{0.3}$
Entropy bonus	0.01	none (the magnet takes its role)
Learning rate	2.5×10^{-4}, linear anneal	power-law, coordinated with τ
Advantage filtering	no	top quarter by advantage
Advantage normalization	yes	no
GAE λ; discount γ	0.95; 0.99	0.5 policy, 0.8 value; 1
Value head	scalar, MSE	categorical win/draw/loss, cross-entropy
Epochs per batch	4	1
Evaluation weights	raw	EMA 0.999 available

Table 2: Our earlier PPO recipe and the adapted Ataraxos move recipe.

Four ways to limit change. The learning rate, ratio clipping and the two KL terms all slow the policy down, but they measure different things.

Knob	Measures change in	Relative to	Main job
Learning rate	parameters	nothing	optimization speed
Ratio clipping	sampled action's probability	π_{old}	cap any one sample's pull
$\mathrm{KL}(\pi \,\|\, \pi_{\text{old}})$	whole action distribution	π_{old}	trust region on this batch
$\mathrm{KL}(\pi \,\|\, \rho)$	whole action distribution	fixed magnet	keep strategic breadth

Clipping covers only the sampled action. Once a sample's ratio passes $1 \pm \epsilon$, its gradient goes to zero, but nothing pulls the policy back if other samples push it further. The KL to π_{old} covers every legal action and acts as a restoring force. Computing it requires the full collection distribution, so the collector stores that distribution for every row. The magnet KL has the same form, but its anchor is fixed: the KL to π_{old} limits speed, and the magnet KL limits where the policy can end up. Our magnet is uniform over action types, then uniform within a type. A spell with dozens of target encodings therefore gets no more prior weight than a single attack. In Stratego, the corresponding reference picks a movable piece uniformly, then one of its moves.

The energy reserve. The view we took from Ataraxos is that a strong early pull toward a broad reference keeps alternative lines alive long enough for the opponent to teach their value. Annealing it too fast gives quick early gains, then entropy collapse, stalled learning and exploitability. Annealing it too slowly leaves the policy diffuse. This is not a claim that entropy equals strength; random mana payments raise entropy without strategic mixing. It is a claim about keeping the capacity to adapt. Measuring it needs entropy by decision class, such as $P(\text{pass} \mid \text{instant in hand, mana open})$. Today we log only global entropy from the last minibatch.

Off-policy data. The policy gradient averages over states and actions that the *current* policy would produce. Data from an older policy is wrong in four ways:

- **Actions.** They need importance ratios, which have high variance.
- **States.** Correcting the state distribution exactly requires ratio products over the whole game. Per-step ratios averaging 1.05 over 300 decisions would multiply to about 2×10^6.
- **Advantages.** They were computed under the old policy's continuation.

- **Opponent.** In self-play the opponent in the data is stale too.

Value learning that combines bootstrapping, function approximation and off-policy data can diverge [Sutton and Barto, 2018]. We keep the policy close to the collecting policy instead of correcting the state distribution: weights are frozen per batch, there is one epoch and no replay buffer, and clipping, the collection KL, gradient-norm clipping and an annealed learning rate bound each update. Collection probabilities are saved per row and checked against the selected-action likelihoods. Advantage filtering deliberately changes which states the policy learns from.

Departures from the source. Following `docs/ataraxos.md`: learner decisions replace Stratego's same-player positions; MTG action types replace the piece-then-move reference groups; CPU learner streams replace large GPU simulator batches; and quantile interpolation, Adam constants and the iteration origin were unspecified in the source, so our choices are explicit rather than recovered.

10 Beliefs and local search

Why not poker methods. CFR and its descendants solve for a strategy over every private state a player could hold [Zinkevich et al., 2007, Brown and Sandholm, 2018]. Public-belief-state methods turn that into a search with well-defined values [Moravčík et al., 2017, Brown et al., 2020, Schmid et al., 2023]. They are the correct theory, but expensive here. Today's opponent hands are poker-sized: 10,832 seven-card hands, against 1,326 in hold'em. That number grows to about 10^7 at 40 distinct card names, before library order. Actions are variable and combinatorial, and games last hundreds of decisions. Stratego shares *Magic*'s huge hidden state and long games, and DeepNash and Ataraxos succeeded there with regularized self-play rather than belief solving. That made them the closer precedent.

Search as an inference-time correction. Ataraxos reverses the KataGo arrangement: self-play teaches the policy, and search makes a small correction at inference. A belief network, trained by supervised learning on the true hidden state from self-play games, samples hidden positions. A frozen policy plays depth-limited rollouts, a value network scores them, and the averaged values $q(a)$ feed one magnetic mirror descent step [Sokota et al., 2023]:

$$\pi^+(a) \;\propto\; \pi_0(a)^{\frac{1}{1+\eta\alpha}}\;\rho(a)^{\frac{\eta\alpha}{1+\eta\alpha}}\;\exp\Big(\tfrac{\eta}{1+\eta\alpha}\,q(a)\Big).$$

This is the maximizer of $\sum_a \pi(a)q(a) - \alpha\mathrm{KL}(\pi\|\rho) - \eta^{-1}\mathrm{KL}(\pi\|\pi_0)$. Each KL becomes a log-probability term weighted by its strength, so the update starts from the raw policy π_0, blends in the magnet ρ, and tilts toward higher value. It anchors noisy rollouts to what self-play learned and keeps the policy mixed, where an argmax would not. The paper writes the logits as $(q+\alpha\log\rho+\beta\log\pi_0)/(\alpha+\beta)$ with $\beta = 1/\eta$, and reports $\alpha = 0.002$, $\beta = 0.02$, 1,000 rollouts and depth 40. Our defaults are $\alpha = 0.02$, $\beta = 1$, depth 2 and one sampled world. With these, q is multiplied by about 0.98 in the logits, against about 45 in the paper. Unless our values are on a much larger scale — which we have not checked — our search barely moves the policy.

Our continuations act from each simulated player's own observation, and values are averaged across sampled worlds before an action is chosen. The value network sees only the acting player's observation, but is trained on outcomes that depend on the true hidden state. It therefore estimates the value of an information state, averaged over hidden states in proportion to how often self-play produces them. Belief sampling and local search run end to end: 840 held-out sampled hands

had no support violations, and 183 search commands replayed with no mismatch. Neither has yet improved on the policy alone. Distilling the *regularized* search distribution back into the policy is not part of Ataraxos; for us it is an open experiment.

Strategy fusion: suspected, not isolated. Searching sampled hidden worlds as if they were face-up has two known flaws [Frank and Basin, 1998, Long et al., 2010]. *Strategy fusion* arises when the search makes the player's *later* choices inside each sampled world: it credits the player with a different perfect plan in each world, which no real player can follow. A continuation policy that sees only its own player's observation cannot do this. Neither can one-ply search with random rollouts, which is what our flat search used. Separately, if the simulated opponent sees the full state, bluffing is worth nothing in every sampled world. Our July draft attributed the hold-the-wipe failure to strategy fusion. A later review judged that premature: random continuations rarely execute "wait, then wipe" even with everything face-up, so rollout pathology is the more likely cause. We designed a two-by-two test — hidden versus face-up information, crossed with random versus strong continuations — but have not run it. A preregistered tripwire for fusion, an exploiter reaching at least 65% against the distilled student, did not fire; the best attacker reached 29.8%.

11 Evaluation

Three kinds of progress are easy to confuse: *optimization* progress (loss, KL, entropy), *relative* progress (wins against the latest opponent), and *durable* progress (strength across past, varied and held-out opponents). The first two can improve while the third gets worse. Our practice follows from this:

- **Balanced play.** Every evaluation balances seats and decks, because between uniformly random players on a creature deck, going first wins 94% of games.
- **Head-to-head Tournaments.** A Tournament file names a roster of checkpoints and fixed opponents, and plays every pairing on the same held-out deals with both decks and both seats. This separates checkpoints that look the same against a scripted opponent. The arena fits ratings across the whole population.
- **Anchors.** Random and scripted-greedy anchors are saturating. We are testing whether a rule-based heuristic bot and frozen trained checkpoints can rank checkpoints of different strength.
- **Replay and seeds.** Every game is replay-verified. Claims about a training method use independent training seeds, because game-level intervals only measure one checkpoint's evaluation noise.
- **Preregistration.** The question, numeric prediction, kill criteria and cost cap are committed before the run. Refuted predictions stay on the record.

12 Open questions

- **Curriculum.** Which matchups (mirrors or cross-deck), which opponents, how many decks? The energy reserve keeps options alive; the curriculum decides which experiences get to use them. Ataraxos did little curriculum design.
- **The magnet.** Uniform over action types, uniform over complete actions, a heuristic bot, or a human prior? What test shows that it improves robustness, not just entropy?

- **Do the pieces only matter together, or only at scale?** How can we tell cheaply?
- **Exploration.** Is a missing behavior unexplored, unrepresentable, or unrewarded by current opponents?
- **Search.** Does inference-time search help a strong policy here, and is distilling regularized search targets the KataGo move for *Magic*, or a return of the label-cost problem?
- **Representation.** Which auxiliary targets improve strength per environment step, and do typed card programs transfer to held-out cards?
- **Evaluation.** What evidence would show a stronger mixed strategy, not a better response to one evaluation set, and how do we compare against humans?

13 Limitations

The rules subset omits replacement and prevention effects, full layers and mulligans. The card pool is two decks. Opponents are scripted bots and our own checkpoints; there is no human evaluation. Several self-play findings rest on two or three training seeds. The literature mapping reflects our reading and is not a survey.

Reproducibility. Every number traces to a report under `experiments/`. The research summary in `manabot/RESEARCH.md` links current findings to their reports. Training recipes are JSON regimes under `experiments/regimes/`; the current baseline is `batched-baseline.json`.

A Difficulties of game RL, mapped to the literature

For readers who want the problem-first framing: each row names a difficulty, how it showed up in Manabot, the techniques that address it, and our current choice.

Table 3: Difficulties of self-play RL for games, the techniques that address them, and Manabot's current choices.

Difficulty	How it showed up	Techniques (origin)	Manabot now
Simulation and engine time	Engine moved from a Python prototype to C++ to Rust. Engine now ≈4.5% of collection; policy calls ≈87%; ≈11 positions per call across 64 games	Fast exact simulators with clone/fork [Silver et al., 2018, Wu, 2019]; vectorized environments and decoupled actors/learners [Mnih et al., 2016, Espeholt et al., 2018]; GPU simulation [Makoviychuk et al., 2021, Sokota et al., 2026]; search on only some moves [Wu, 2019]	Rust engine and vector env, zero-copy observations, 64–256 batched games. A GPU engine would save at most ≈4.5%; the next levers are inference batching and dispatch
Credit assignment	Shaped rewards lost about 15 points to terminal-only reward	TD(λ) [Sutton, 1988]; GAE [Schulman et al., 2016]; search-derived targets [Silver et al., 2017, 2018]	Terminal reward only; GAE $\lambda = 0.5$ for the policy and 0.8 for the value; categorical win/draw/loss value head
Representation	Open: does the card encoder learn mechanics, or memorize a two-deck pool?	Auxiliary prediction targets [Jaderberg et al., 2017, Wu, 2019]; generalized card representations [Bertram et al., 2024]	Typed card-program tokens through a GRU, plus object attention; no auxiliary targets; held-out-card tests planned

Difficulty	How it showed up	Techniques (origin)	Manabot now
Exploration	The search pilot almost never bluffed or held up instant-speed interaction	Entropy bonus [Williams and Peng, 1991]; KL to a broad reference [Perolat et al., 2022, Sokota et al., 2023]; root noise in search [Silver et al., 2017]; varied start states [Ecoffet et al., 2021]; leagues and exploiters [Vinyals et al., 2019]	Sampling from the policy plus an action-type-uniform magnet; no opponent pool; scenario starts and exploiters exist but are used only for evaluation
Instability and degeneracy	One round of expert iteration lost 74–26 to round 0	Trust regions and ratio clipping [Schulman et al., 2015, 2017]; KL to the collecting policy; magnet with coordinated annealing, and weight averaging for evaluation [Sokota et al., 2026]	Ataraxos move recipe in the baseline. No single piece has shown a clear benefit yet (§2)
Off-policy data	Collection freezes weights per batch, but games span updates	Clipped importance ratios [Schulman et al., 2017]; V-trace [Espeholt et al., 2018]; Retrace [Munos et al., 2016]; target re-analysis [Schrittwieser et al., 2021]	Clip 0.2, collection KL 0.1, one epoch per batch, no replay buffer (§9)
Cycling	Changing only the scripted opponent's deck moved Lessons results by 40–60 points	Fictitious play [Brown, 1951, Heinrich and Silver, 2016]; PSRO [Lanctot et al., 2017]; leagues [Vinyals et al., 2019]; regularized last-iterate dynamics [Perolat et al., 2021, Sokota et al., 2023]; CFR in small games [Zinkevich et al., 2007]	Current-self opponent only, with magnet and EMA weights; no population yet
Hidden information	The search pilot failed control scenarios. Strategy fusion was suspected but never isolated (§10)	Critique of determinization [Frank and Basin, 1998, Long et al., 2010]; information-set search [Cowling et al., 2012a]; public belief states [Moravčík et al., 2017, Brown et al., 2020, Schmid et al., 2023]; sampled beliefs plus one regularized update [Sokota et al., 2026]	Belief sampler and local search run end to end; no strength gain over the policy alone yet
Evaluation	99% against random while failing control scenarios; round 1 lost to round 0	Population ratings and Nash averaging [Balduzzi et al., 2018]; exploiters [Vinyals et al., 2019]; behavioral probes	Arena ratings, head-to-head Tournaments, competency scenarios. Scripted opponents are saturating; human comparison not yet built

PPO's ratio *clipping* is the defense against off-policy data; *advantage filtering* decides which samples are worth learning from. Ataraxos uses both.

The July 2026 draft (`drafts/02-platform-and-challenges.tex`) keeps the full search-era tables and calibration ledger.

References

David Balduzzi, Karl Tuyls, Julien Perolat, and Thore Graepel. Re-evaluating evaluation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Timo Bertram, Johannes Fürnkranz, and Martin Müller. Learning with generalised card representations for Magic: The Gathering. In *IEEE Conference on Games (CoG)*, 2024.

George W. Brown. Iterative solution of games by fictitious play. In *Activity Analysis of Production and Allocation*. Wiley, 1951.

Noam Brown and Tuomas Sandholm. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424, 2018.

Noam Brown, Anton Bakhtin, Adam Lerer, and Qucheng Gong. Combining deep reinforcement learning and search for imperfect-information games. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Peter I. Cowling, Edward J. Powley, and Daniel Whitehouse. Information set monte carlo tree search. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2):120–143, 2012a.

Peter I. Cowling, Colin D. Ward, and Edward J. Powley. Ensemble determinization in monte carlo tree search for the imperfect information card game Magic: The Gathering. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(4):241–257, 2012b.

Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590:580–586, 2021.

Lasse Espeholt et al. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *International Conference on Machine Learning (ICML)*, 2018.

Ian Frank and David Basin. Search in games with incomplete information: A case study using bridge card play. *Artificial Intelligence*, 100(1–2):87–123, 1998.

Johannes Heinrich and David Silver. Deep reinforcement learning from self-play in imperfect-information games. *arXiv preprint arXiv:1603.01121*, 2016.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

Max Jaderberg, Volodymyr Mnih, Wojciech Marian Czarnecki, Tom Schaul, Joel Z. Leibo, David Silver, and Koray Kavukcuoglu. Reinforcement learning with unsupervised auxiliary tasks. In *International Conference on Learning Representations (ICLR)*, 2017.

Marc Lanctot et al. A unified game-theoretic approach to multiagent reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Jeffrey Richard Long, Nathan R. Sturtevant, Michael Buro, and Timothy Furtak. Understanding the success of perfect information monte carlo sampling in game tree search. In *Proceedings of the 24th AAAI Conference on Artificial Intelligence*, 2010.

Viktor Makoviychuk et al. Isaac Gym: High performance GPU-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021.

Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2016.

Matej Moravčík et al. DeepStack: Expert-level artificial intelligence in heads-up no-limit poker. *Science*, 356(6337):508–513, 2017.

Rémi Munos, Tom Stepleton, Anna Harutyunyan, and Marc G. Bellemare. Safe and efficient off-policy reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

Julien Perolat, Remi Munos, Jean-Baptiste Lespiau, Shayegan Omidshafiei, Mark Rowland, Pedro Ortega, Neil Burch, Thomas Anthony, David Balduzzi, Bart De Vylder, Georgios Piliouras, Marc Lanctot, and Karl Tuyls. From Poincaré recurrence to convergence in imperfect information games: Finding equilibrium via regularization. In *International Conference on Machine Learning (ICML)*, 2021. verify.

Julien Perolat et al. Mastering the game of Stratego with model-free multiagent reinforcement learning. *Science*, 378(6623):990–996, 2022. verify.

Martin Schmid et al. Student of games: A unified learning algorithm for both perfect and imperfect information games. *Science Advances*, 9(46), 2023. verify.

Julian Schrittwieser, Thomas Hubert, Amol Mandhane, Mohammadamin Barekatain, Ioannis Antonoglou, and David Silver. Online and offline reinforcement learning by planning with a learned model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. verify.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*, 2015.

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

David Silver et al. Mastering the game of Go without human knowledge. *Nature*, 550:354–359, 2017.

Samuel Sokota, Ryan D'Orazio, J. Zico Kolter, Nicolas Loizou, Marc Lanctot, Ioannis Mitliagkas, Noam Brown, and Christian Kroer. A unified approach to reinforcement learning, quantal response equilibria, and two-player zero-sum games. In *International Conference on Learning Representations (ICLR)*, 2023. verify.

Samuel Sokota et al. [Title to confirm] Ataraxos: superhuman Stratego from damped self-play. *Nature*, 2026. doi: 10.1038/s41586-026-11036-y. verify: title and full author list.

Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3:9–44, 1988.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.

Ronald J. Williams and Jing Peng. Function optimization using connectionist reinforcement learning algorithms. *Connection Science*, 3(3):241–268, 1991.

David J. Wu. Accelerating self-play learning in Go. *arXiv preprint arXiv:1902.10565*, 2019.

Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2007.